

Dictionaries in Python

A dictionary in Python is a collection of **key-value pairs**. Each key in a dictionary is associated with a value, and you can retrieve or manipulate data using the key. Unlike lists and tuples, dictionaries are **unordered** and **mutable** (changeable).

1. Creating a Dictionary

You can create a dictionary using curly braces {} or the dict() function.

Syntax:

```
my_dict = {  
    "key1": "value1",  
    "key2": "value2",  
    "key3": "value3"  
}
```

Example:

Let's create a dictionary of famous cities in Karnataka and their popular dishes.

```
karnataka_food = {  
    "Bengaluru": "Bisi Bele Bath",  
    "Mysuru": "Mysore Pak",  
    "Mangaluru": "Neer Dosa"  
}
```

2. Accessing Dictionary Elements

To access the values stored in a dictionary, you use the key.

Example:

```
print(karnataka_food["Mysuru"]) # Output: Mysore Pak
```

You can also use the get() method to access values, which is safer because it doesn't throw an error if the key doesn't exist.

```
print(karnataka_food.get("Mangaluru")) # Output: Neer Dosa
```

```
print(karnataka_food.get("Shivamogga", "Not Found")) # Output: Not Found
```

3. Adding and Updating Dictionary Elements

You can add new key-value pairs or update existing values in a dictionary.

Adding an Item:

```
karnataka_food["Shivamogga"] = "Kadubu"  
print(karnataka_food)
```

Updating an Item:

```
karnataka_food["Bengaluru"] = "Ragi Mudde"
```

4. Removing Elements from a Dictionary

You can remove items from a dictionary using several methods:

- `pop()`: Removes the specified key and returns the associated value.
- `mysuru_food = karnataka_food.pop("Mysuru")`

```
print(mysuru_food) # Output: Mysore Pak
```

- `del`: Removes the specified key.

```
del karnataka_food["Mangaluru"]
```

- `clear()`: Empties the dictionary.

```
karnataka_food.clear()
```

5. Dictionary Methods

Here are some common methods available for dictionaries:

- `keys()`: Returns all the keys in the dictionary.

```
print(karnataka_food.keys()) # Output: dict_keys(['Bengaluru', 'Mysuru', 'Mangaluru'])
```

- `values()`: Returns all the values in the dictionary.

```
print(karnataka_food.values()) # Output: dict_values(['Bisi Bele Bath', 'Mysore Pak', 'Neer Dosa'])
```

- `items()`: Returns key-value pairs as tuples.

```
print(karnataka_food.items()) # Output: dict_items([('Bengaluru', 'Bisi Bele Bath'), ('Mysuru', 'Mysore Pak'), ('Mangaluru', 'Neer Dosa')])
```

- `update()`: Updates the dictionary with another dictionary or iterable.
- `new_dishes = {"Hubballi": "Girmit"}`

```
karnataka_food.update(new_dishes)
```

6. Dictionary Characteristics

- **Unordered**: Dictionary keys are not stored in any particular order.
- **Mutable**: You can change, add, or remove items.
- **Keys Must Be Immutable**: Keys in a dictionary must be of a data type that is immutable, such as a string, number, or tuple.
- **Unique Keys**: A dictionary cannot have duplicate keys. If you try to add a duplicate key, the latest value will overwrite the previous one.

7. Differences Between Lists, Tuples, Sets, and Dictionaries

Feature	List	Tuple	Set	Dictionary
Ordering	Ordered	Ordered	Unordered	Unordered
Mutability	Mutable	Immutable	Mutable	Mutable
Duplicates	Allows duplicates	Allows duplicates	No duplicates	Keys: No duplicates
Indexing	Supports indexing	Supports indexing	No indexing	Uses keys
Structure	Indexed collection	Indexed collection	Unordered collection	Key-value pairs